

15. Linear regression

The goal in this chapter is to introduce **linear regression**, the standard tool that statisticians rely on when analysing the relationship between interval scale predictors and interval scale outcomes. Stripped to its bare essentials, linear regression models are basically a slightly fancier version of the Pearson correlation (Section 5.7) though as we'll see, regression models are much more powerful tools.

15.1

What is a linear regression model?

Since the basic ideas in regression are closely tied to correlation, we'll return to the `parenthood.Rdata` file that we were using to illustrate how correlations work. Recall that, in this data set, we were trying to find out why Dan is so very grumpy all the time, and our working hypothesis was that I'm not getting enough sleep. We drew some scatterplots to help us examine the relationship between the amount of sleep I get, and my grumpiness the following day. The actual scatterplot that we draw is the one shown in Figure 15.1, and as we saw previously this corresponds to a correlation of $r = -.90$, but what we find ourselves secretly imagining is something that looks closer to Figure 15.2a. That is, we mentally draw a straight line through the middle of the data. In statistics, this line that we're drawing is called a **regression line**. Notice that – since we're not idiots – the regression line goes through the middle of the data. We don't find ourselves imagining anything like the rather silly plot shown in Figure 15.2b.

This is not highly surprising: the line that I've drawn in Figure 15.2b doesn't "fit" the data very well, so it doesn't make a lot of sense to propose it as a way of summarising the data, right? This is a very simple observation to make, but it turns out to be very powerful when we start trying to wrap just a little bit of maths around it. To do so, let's start with a refresher of some high school maths. The formula for a straight line is usually written like this:

$$y = mx + c$$

Or, at least, that's what it was when I went to high school all those years ago. The two *variables* are x and y , and we have two *coefficients*, m and c . The coefficient m represents the *slope* of the line, and the coefficient c represents the *y-intercept* of the line. Digging further back into our decaying memories of high school (sorry, for some of us high school was a long time ago), we remember that the intercept is interpreted as "the value of y that you get when $x = 0$ ". Similarly, a slope of m means that if you increase the x -value by 1 unit, then the y -value goes up by m units; a negative slope means that the y -value would go down rather than up. Ah yes, it's all coming back to me now.

Now that we've remembered that, it should come as no surprise to discover that we use the exact same formula to describe a regression line. If Y is the outcome variable (the DV) and X is the predictor

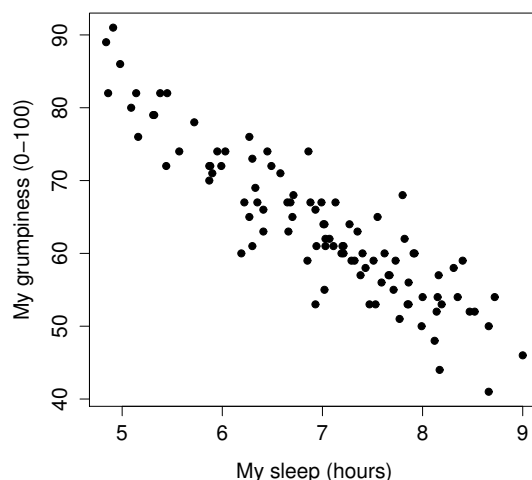


Figure 15.1: Scatterplot showing grumpiness as a function of hours slept.

variable (the IV), then the formula that describes our regression is written like this:

$$\hat{Y}_i = b_1 X_i + b_0$$

Hm. Looks like the same formula, but there's some extra frilly bits in this version. Let's make sure we understand them. Firstly, notice that I've written X_i and Y_i rather than just plain old X and Y . This is because we want to remember that we're dealing with actual data. In this equation, X_i is the value of predictor variable for the i th observation (i.e., the number of hours of sleep that I got on day i of my little study), and Y_i is the corresponding value of the outcome variable (i.e., my grumpiness on that day). And although I haven't said so explicitly in the equation, what we're assuming is that this formula works for all observations in the data set (i.e., for all i). Secondly, notice that I wrote $\hat{Y}_i$ and not Y_i . This is because we want to make the distinction between the *actual data* Y_i , and the *estimate* $\hat{Y}_i$ (i.e., the prediction that our regression line is making). Thirdly, I changed the letters used to describe the coefficients from m and c to b_1 and b_0 . That's just the way that statisticians like to refer to the coefficients in a regression model. I've no idea why they chose b , but that's what they did. In any case b_0 always refers to the intercept term, and b_1 refers to the slope.

Excellent, excellent. Next, I can't help but notice that – regardless of whether we're talking about the good regression line or the bad one – the data don't fall perfectly on the line. Or, to say it another way, the data Y_i are not identical to the predictions of the regression model $\hat{Y}_i$. Since statisticians love to attach letters, names and numbers to everything, let's refer to the difference between the model prediction and that actual data point as a *residual*, and we'll refer to it as ϵ_i .¹ Written using mathematics, the residuals are defined as:

$$\epsilon_i = Y_i - \hat{Y}_i$$

which in turn means that we can write down the complete linear regression model as:

$$Y_i = b_1 X_i + b_0 + \epsilon_i$$

¹The ϵ symbol is the Greek letter epsilon. It's traditional to use ϵ_i or e_i to denote a residual.

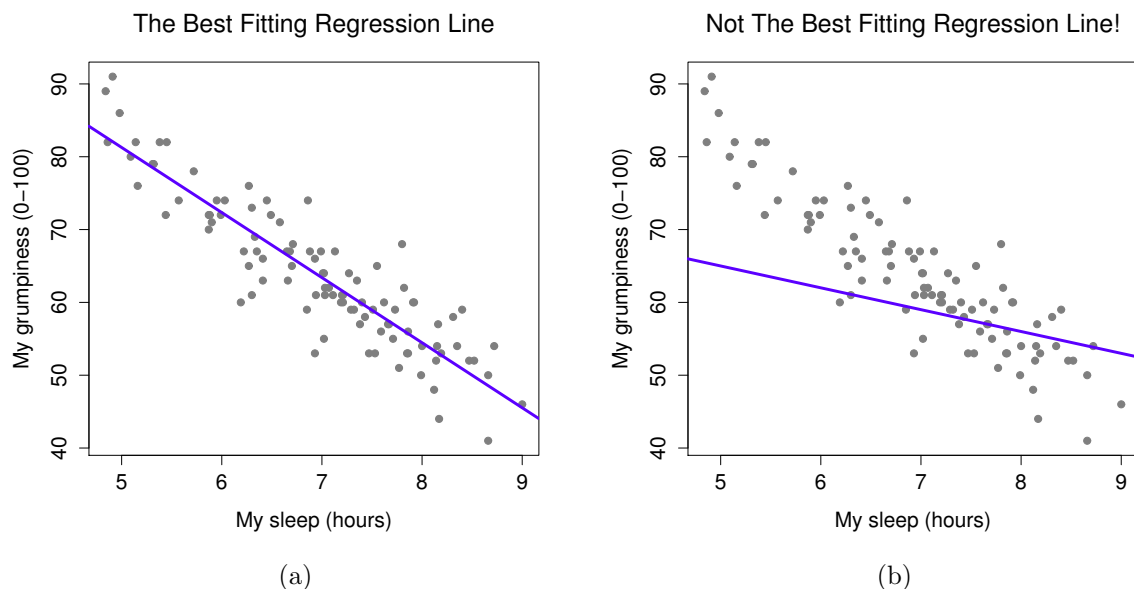


Figure 15.2: Panel a shows the sleep-grumpiness scatterplot from Figure 15.1 with the best fitting regression line drawn over the top. Not surprisingly, the line goes through the middle of the data. In contrast, panel b shows the same data, but with a very poor choice of regression line drawn over the top.

15.2

Estimating a linear regression model

Okay, now let's redraw our pictures, but this time I'll add some lines to show the size of the residual for all observations. When the regression line is good, our residuals (the lengths of the solid black lines) all look pretty small, as shown in Figure 15.3a, but when the regression line is a bad one, the residuals are a lot larger, as you can see from looking at Figure 15.3b. Hm. Maybe what we “want” in a regression model is *small* residuals. Yes, that does seem to make sense. In fact, I think I'll go so far as to say that the “best fitting” regression line is the one that has the smallest residuals. Or, better yet, since statisticians seem to like to take squares of everything why not say that ...

The estimated regression coefficients, $\hat{b}_0$ and $\hat{b}_1$ are those that minimise the sum of the squared residuals, which we could either write as $\sum_i (Y_i - \hat{Y}_i)^2$ or as $\sum_i \epsilon_i^2$.

Yes, yes that sounds even better. And since I've indented it like that, it probably means that this is the right answer. And since this is the right answer, it's probably worth making a note of the fact that our regression coefficients are *estimates* (we're trying to guess the parameters that describe a population!), which is why I've added the little hats, so that we get $\hat{b}_0$ and $\hat{b}_1$ rather than b_0 and b_1 . Finally, I should also note that – since there's actually more than one way to estimate a regression model – the more technical name for this estimation process is **ordinary least squares (OLS) regression**.

At this point, we now have a concrete definition for what counts as our “best” choice of regression coefficients, $\hat{b}_0$ and $\hat{b}_1$. The natural question to ask next is, if our optimal regression coefficients are those that minimise the sum squared residuals, how do we *find* these wonderful numbers? The actual answer

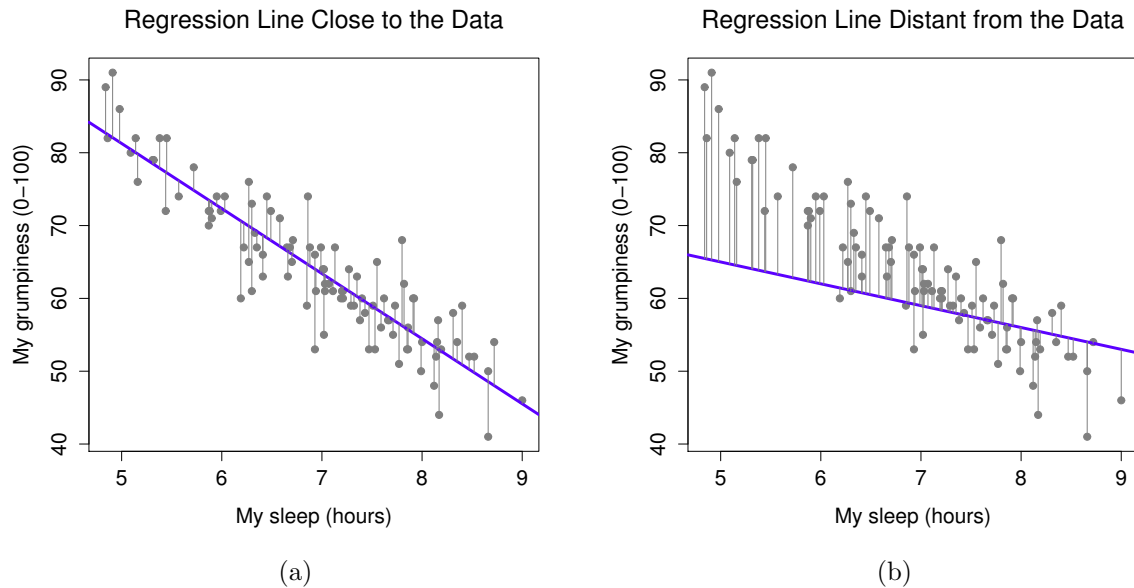


Figure 15.3: A depiction of the residuals associated with the best fitting regression line (panel a), and the residuals associated with a poor regression line (panel b). The residuals are much smaller for the good regression line. Again, this is no surprise given that the good line is the one that goes right through the middle of the data.

to this question is complicated, and it doesn't help you understand the logic of regression.² As a result, this time I'm going to let you off the hook. Instead of showing you how to do it the long and tedious way first, and then "revealing" the wonderful shortcut that R provides you with, let's cut straight to the chase... and use the `lm()` function (short for "linear model") to do all the heavy lifting.

15.2.1 Using the `lm()` function

The `lm()` function is a fairly complicated one: if you type `?lm`, the help files will reveal that there are a lot of arguments that you can specify, and most of them won't make a lot of sense to you. At this stage however, there's really only two of them that you care about, and as it turns out you've seen them before:

- **formula.** A formula that specifies the regression model. For the simple linear regression models that we've talked about so far, in which you have a single predictor variable as well as an intercept term, this formula is of the form `outcome ~ predictor`. However, more complicated formulas are allowed, and we'll discuss them later.

²Or at least, I'm assuming that it doesn't help most people. But on the off chance that someone reading this is a proper kung fu master of linear algebra (and to be fair, I always have a few of these people in my intro stats class), it *will* help *you* to know that the solution to the estimation problem turns out to be $\hat{\mathbf{b}} = (\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{y}$, where $\hat{\mathbf{b}}$ is a vector containing the estimated regression coefficients, $\mathbf{X}$ is the "design matrix" that contains the predictor variables (plus an additional column containing all ones; strictly $\mathbf{X}$ is a matrix of the regressors, but I haven't discussed the distinction yet), and $\mathbf{y}$ is a vector containing the outcome variable. For everyone else, this isn't exactly helpful, and can be downright scary. However, since quite a few things in linear regression can be written in linear algebra terms, you'll see a bunch of footnotes like this one in this chapter. If you can follow the maths in them, great. If not, ignore it.

- `data`. The data frame containing the variables.

As we saw with `aov()` in Chapter 14, the output of the `lm()` function is a fairly complicated object, with quite a lot of technical information buried under the hood. Because this technical information is used by other functions, it's generally a good idea to create a variable that stores the results of your regression. With this in mind, to run my linear regression, the command I want to use is this:

```
> regression.1 <- lm( formula = dan.grump ~ dan.sleep,
+                     data = parenthood )
```

Note that I used `dan.grump ~ dan.sleep` as the formula: in the model that I'm trying to estimate, `dan.grump` is the *outcome* variable, and `dan.sleep` is the predictor variable. It's always a good idea to remember which one is which! Anyway, what this does is create an "lm object" (i.e., a variable whose class is "lm") called `regression.1`. Let's have a look at what happens when we `print()` it out:

```
> print( regression.1 )

Call:
lm(formula = dan.grump ~ dan.sleep, data = parenthood)

Coefficients:
(Intercept)    dan.sleep
    125.956         -8.937
```

This looks promising. There's two separate pieces of information here. Firstly, R is politely reminding us what the command was that we used to specify the model in the first place, which can be helpful. More importantly from our perspective, however, is the second part, in which R gives us the intercept $\hat{b}_0 = 125.96$ and the slope $\hat{b}_1 = -8.94$. In other words, the best-fitting regression line that I plotted in Figure 15.2 has this formula:

$$\hat{Y}_i = -8.94 X_i + 125.96$$

15.2.2 Interpreting the estimated model

The most important thing to be able to understand is how to interpret these coefficients. Let's start with $\hat{b}_1$, the slope. If we remember the definition of the slope, a regression coefficient of $\hat{b}_1 = -8.94$ means that if I increase X_i by 1, then I'm decreasing Y_i by 8.94. That is, each additional hour of sleep that I gain will improve my mood, reducing my grumpiness by 8.94 grumpiness points. What about the intercept? Well, since $\hat{b}_0$ corresponds to "the expected value of Y_i when X_i equals 0", it's pretty straightforward. It implies that if I get zero hours of sleep ($X_i = 0$) then my grumpiness will go off the scale, to an insane value of ($Y_i = 125.96$). Best to be avoided, I think.

15.3

Multiple linear regression

The simple linear regression model that we've discussed up to this point assumes that there's a single predictor variable that you're interested in, in this case `dan.sleep`. In fact, up to this point, *every* statistical tool that we've talked about has assumed that your analysis uses one predictor variable and one outcome variable. However, in many (perhaps most) research projects you actually have multiple predictors that you want to examine. If so, it would be nice to be able to extend the linear regression